FIND THE MOMENT. KEEP THE QUESTION.

Cut Before.

A practical promo finder for a busy story editor.

50 episodes read | 17 test first | 20 review further | 13 episodes not recommended

Atit Kumar Satsangi | Roll: 230241 | Civil Engineering

The problem I chose

Problem 1: Find the Promos. An editor needs the right cut, not another plot summary. This problem has real scripts and a result we can check. The idea is simple: **give the viewer enough to care, then cut before the answer.**

The scope

One continuous 20-60 second clip per selected episode. No new dialogue, music or overlays. The supplied timestamps are treated as episode-relative script times; an editor must check the actual footage. Quiet episodes can be rejected. This version does not build montages.

How it works

1. Code finds valid windows. It reads every timed event, checks the input format and suggests up to three varied clips per episode using simple story signals.
2. AI judges the story. A separate chat review sees the timelines and suggestions. It can change the cut, flag it for more review or reject the episode. It checks the opening, ending and what happens next.
3. Code checks the answer. It rejects invented boundaries, cuts through spoken lines, wrong-input reviews and clips outside 20-60 seconds. It exports a searchable report and CSV edit list.

The first three clips I would test

01 / Crawl, or your husband hangs
EP 12 | 00:44.00 - 01:09.20 | 25.2s
A sister demands that Juliana crawl into thorns. The cut ends on the threat to hang her husband. A clear choice with a personal cost; test this first.

02 / She found the poison. Someone found her.
EP 23 | 01:19.00 - 02:04.00 | 45.0s
"Someone is poisoning her" opens the clip. It ends as someone approaches the room. Test this next for mystery, though the preparation scene may feel slow.

03 / The ceiling is coming down
EP 18 | 00:39.50 - 01:08.00 | 28.5s
They cannot escape the fire, and she will not leave a book. End on the warning about the ceiling. Test a visual danger hook after the two dialogue-led ideas.

Useful. Checkable. Honest.

What the real run showed

The code read **50/50 episodes and 3,727 events**, checked 15,152 valid windows and returned 149 rule suggestions. The saved AI review covers all 50 episodes. All **37 retained clips** pass the duration, exact-boundary and uncut-dialogue checks. The 12 code tests also pass, including a new story about a flooded vault. These checks establish technical validity, not ad quality.

Where the tool picked badly

The AI-written rule scorer ranked episode 11, **00:16.80-00:37.10**, second overall. Its theft accusation is a hook, but it ends on another accusation: "He threatens me again!" The review moved the ending to **00:53.60**, when Alexander allows the search. Now the open question is what the constable will find.
In a self-audit of the baseline's top ten, the AI kept only **2 unchanged**, flagged 5 for more review and rejected 3. This is the same assistant checking its work, not independent accuracy testing. The raw baseline and audit are included.

Ranking and human judgment

The review scores hook, unresolved ending, standalone clarity and stakes from 0-5. Weights are **30%, 35%, 25% and 10%**. Test first means stronger ideas; review further means story or pacing concerns. The first group ranks ahead of the second. Scores are opinions, not predicted clicks. An editor still checks the first three seconds, pacing, spoilers, violence and picture cuts. Some scripts pack too many words into a short time; the tool flags this.

How I would know it helps

Ask two editors to judge 30 suggestions without reading the earlier story. Track how many they keep, how much they change the boundaries and time spent per usable clip. Then test the strongest clips with the same audience, budget and placement. Track three-second retention and cost per new viewer who finishes episode one. No audience uplift has been measured yet.

Run again, change the model, scale up

The local Python tool needs no extra packages or API key. New inputs produce new review packets. Any approved model can return the same JSON; the code checks it before ranking. **The model step is manual in this version.** At scale, cache unchanged episodes, batch reviews through an approved API and send the strongest ideas to an editor. Model tokens and editor time drive cost; no API cost or scale benchmark is claimed.

AI use and delivery

Tools used: **Codex** for code, story review and writing; **Python** for parsing and checks; **ReportLab** for this PDF. The code makes no model calls. The episode 11 error above shows why AI suggestions need review. Delivery includes source code, a notebook with an original demo, the saved review and unedited generated outputs. Kuku scripts are excluded from the public prototype.